Mapping Ghaf Tree Crowns from UAV Imagery

A step-by-step operating handbook

Prosopis cineraria (Ghaf) crown delineation from area-wide UAV orthomosaics, using the trained FastViT-MA36 + Mask2Former model.

Operating instructions for the delivered software bundle.
Each procedure states the command to run, its expected duration, and the output that indicates success.

DOCUMENT	Operating handbook for the Ghaf crown-mapping software
ISSUE	First issue, September 2026
APPLIES TO	The delivered bundle described in section 1, and the repository version recorded in MANIFEST.json
REPOSITORY	github.com/brakuta/Ghaf-Tree-Crown-Mapping-from-UAV-Data
RELATED WORK	Hybrid Vision-CNN Architecture for Mapping *Prosopis cineraria* from Area-wide UAV-based Images
DISTRIBUTION	Issued with the software bundle. The trained weights, the UAV imagery and the labelled tiles are not distributed with the public repository

Contents

1. Folder structure

The delivered folder holds six directories and two files. Paths in this handbook are written as `D:\ghaf-project`. Substitute the location of your own copy. Nothing else changes. Commands are written for the Windows Command Prompt; on macOS and Linux they are identical except that paths use `/` in place of `\`.

1.1 The delivered folder

Listing 1. *Top level*

```
D:\ghaf-project\
+-- README.md                summary of the bundle
+-- MANIFEST.json            inventory of every part, with sizes and checks
+-- code\                    the software (section 1.5)
+-- models\                  six trained models (section 1.2)
+-- init-weights\            ImageNet weights, used only when training
+-- data\ghaf\               labelled tiles (section 1.3)
+-- samples\                 sample orthomosaic for testing the installation
+-- predictions\testing\     predictions already produced for the test split
```

Table 1. *Contents of the delivered folder*

Part	Size	Contents and purpose
`code\`	1 MB	The Python package, the six model configurations, the command-line tools, the test suite and the documentation. This is the public repository, at the exact version recorded in MANIFEST.json
`models\`	2.2 GB	Six trained models. Each folder is self-contained and holds a complete configuration beside its weights
`init-weights\`	0.9 GB	ImageNet initialisation weights for the six backbones. Required only when training a model from scratch on a machine without internet access. Not used for inference
`data\ghaf\`	15.6 GB	8641 labelled tile pairs, divided into training, validation and test splits
`samples\`	0.2 GB	A georeferenced clip of the Kalba survey mosaic, 8192 x 8192 pixels, for confirming that the installation produces correct results
`predictions\testing\`	0.4 GB	Predicted masks for the 767 test tiles, produced with the FastViT model. Provided so that results can be compared without re-running inference
`MANIFEST.json`	12 KB	The inventory. Records each part, its size, its file count, the checks performed at assembly, and the version of the code used. This file identifies which software version produced any given result
`README.md`	4 KB	A one-page summary of the bundle and its origin

1.2 The models folder

Each model occupies one folder of three files. Its configuration is complete in itself and depends on no other file, so a single model folder can be copied elsewhere and used alone.

Listing 2. *Layout of models*

```
models\
+-- fastvit-ma36_mask2former\
|   +-- fastvit-ma36_mask2former.py     the full configuration, self-contained
|   +-- best_mIoU_iter_3500.pth         the trained weights
|   +-- metadata.json                   SHA-256, file size, parameters, scores
+-- poolformer-s36_fpn\
+-- dpn98_fpn\
+-- convnext-small_upernet\
+-- resnet-50_mask2former\
+-- efficientnet-b3_fpn\
+-- MODELS.json                         index of all six, with scores
+-- README.md                           notes for the recipient
```

Table 2. *The six delivered models and their scores on the test split*

Folder	Checkpoint file	mIoU	F1
`fastvit-ma36_mask2former`	best_mIoU_iter_3500.pth	**79.32**	**87.22**
`poolformer-s36_fpn`	iter_10200.pth	78.65	86.72
`dpn98_fpn`	best_mIoU_iter_14000.pth	78.19	86.35
`convnext-small_upernet`	iter_14000.pth	78.02	86.20
`resnet-50_mask2former`	best_mIoU_iter_38500.pth	77.69	85.98
`efficientnet-b3_fpn`	iter_6800.pth	70.77	80.29

Scores are on the held-out test split. Use `fastvit-ma36_mask2former` unless there is a specific reason to prefer another. The remaining five are provided for comparison, and every procedure in this handbook works with any of them by substituting the two paths.

1.3 The data folder

Listing 3. *Layout of data\ghaf*

```
data\ghaf\
+-- training\
|   +-- images\        7005 PNG tiles, 1024 x 1024, 8-bit RGB
|   +-- masks\         7005 PNG masks, one per image, same filename
+-- validation\
|   +-- images\         869 tiles
|   +-- masks\          869 masks
+-- testing\ghaf26\
    +-- images\         767 tiles, with .pgw and .png.aux.xml position files
    +-- masks\          767 masks
```

Table 3. *The three data splits*

Split	Pairs	Purpose
`training`	7 005	Fitting the model
`validation`	869	Monitoring during training and selecting the best checkpoint
`testing/ghaf26`	767	Final scoring. Held out from training entirely, and the source of the published numbers

An image and its mask share a filename. In a mask, the pixel value is the class index rather than a colour: `0` for background and `1` for a Ghaf crown. A mask opened in an image viewer therefore appears almost black, which is correct.

The test tiles carry small companion files, `.pgw`, `.png.aux.xml` and `.ovr`. These record the patch of ground each tile covers and hold display pyramids, so that predictions made from them open in QGIS at the correct location. The training and validation tiles have no companion files, which affects neither training nor scoring. Companion files must be kept beside the tiles they belong to.

1.4 The code folder

The software is organised in four directories. `ghaf\` is the library, `configs\` holds the model recipes, `tools\` holds the programs that are run from the command line, and `tests\` verifies the other three.

Listing 4. *Layout of code*

```
code\
+-- ghaf\                       the library
|   +-- config.py               points a configuration at a dataset folder
|   +-- datasets.py             the two-class tile dataset
|   +-- splits.py               where each split lives, declared once
|   +-- environment.py          confirms the framework is installed correctly
|   +-- init_weights.py         locates the ImageNet initialisation weights
|   +-- release.py              the published models: digests, sizes, scores
|   +-- models\
|   |   +-- fastvit.py          FastViT-MA36 backbone
|   |   +-- dpn.py              DPN-98 backbone
|   |   +-- modules\            building blocks used by FastViT
|   +-- inference\
|       +-- tiling.py           window planning and blending arithmetic
|       +-- large_image.py      orthomosaic inference and georeferenced output
+-- configs\
|   +-- _base_\ghaf.py          dataset, augmentation, schedule and runtime
|   +-- ghaf\                   the six model configurations
+-- tools\                      the command-line programs (table below)
+-- tests\                      325 automated tests
+-- docs\                       this handbook and four reference documents
+-- README.md                   the repository overview
+-- requirements.txt            the Python packages this project adds
+-- environment.yml             the conda environment definition
```

Table 4. *The command-line programs in tools*

Program in tools\	What it does
`smoke_test.py`	Builds all six models, verifies each checkpoint against its published fingerprint, and runs one prediction through each. Section 4
`check_dataset.py`	Verifies that every image has a matching mask, that pairs agree on size, and that masks contain only the two class values. Section 4
`predict_folder.py`	Maps every image in a folder and writes one GeoPackage of crowns per image. Section 6
`test.py`	Scores a model against a labelled split and reports mIoU, mDice and mFscore. Section 8
`predict_split.py`	Writes one predicted mask per tile for a labelled split. Section 8
`train.py`	Trains a model, or fine-tunes one from an existing checkpoint. Section 9
`make_sample.py`	Cuts a small georeferenced clip out of a large mosaic. Section 5

Program in tools\	What it does
`export_release.py`	Assembles the models folder for sharing, verifying every checkpoint before and after copying
`fetch_init_weights.py`	Downloads the ImageNet initialisation weights. Needed once, and only when training
`build_handover.py`	Assembles a complete bundle of the kind described in section 1.1

Area-wide inference is invoked as `python -m ghaf.inference.large_image` rather than through a file in `tools\`, because it forms part of the library. Section 5 gives the command.

1.5 Where results are written

No program writes into `data\`, `models\` or `code\`. Each takes an output path on the command line. The examples in this handbook write to `D:\ghaf-project\output`, which is created on first use.

2. System requirements

Table 5. System requirements

Item	Requirement	Notes
Operating system	Windows 10 or 11, macOS, or Linux	Commands in this handbook are for the Windows Command Prompt
GPU	NVIDIA, 8 GB memory or more	The published models were trained on an RTX A5000. Without a GPU every procedure still runs by adding `--device cpu`, at roughly twenty to fifty times the duration
Disk for the bundle	20 GB	Code, models, initialisation weights, tiles and samples
Disk for one inference run	9 bytes per pixel of the image	An 8192 x 8192 clip requires 0.6 GB. The full 84 072 x 103 691 mosaic requires 79 GB. The space is released when the run ends, and is checked before the run starts
Python	3.9	Installed by conda in section 3. No system-wide Python is involved
Prerequisite software	Miniconda or Anaconda	Available from docs.conda.io. Everything else is installed by the commands in section 3
Optional	QGIS 3.x	For viewing the results. Section 7

Conventions used in this handbook

A shaded box contains a command. Type or paste it, press Enter, and wait for the prompt to return before entering the next one. A box with a green rule shows the output of a correct run. Times and paths will differ. The structure and the reported figures should not.

> **Enter one command at a time.** Some terminals join a multi-line paste into a single line and execute something unintended. A command that wraps onto a second line in this handbook is still one command. Join the parts with a single space when entering it.

3. Installation

Installation takes about twenty minutes and is performed once on each machine. Enter every command in the same window.

Step 1. Open the Anaconda Prompt

Press the Windows key, type Anaconda Prompt, and open it. A window appears with a line ending in >, which is the prompt. Commands are typed after it. A prompt beginning with `(base)` confirms that conda is installed. If no Anaconda Prompt exists on the machine, install Miniconda from docs.conda.io and open a new one.

Step 2. Create the environment

```
conda create -n ghaf python=3.9 -y
```

```
conda activate ghaf
```

> **The prompt now begins with `(ghaf)`.** Every command in this handbook assumes it. A new terminal window always opens outside the environment, so `conda activate ghaf` must be run again in each new window. Omitting it is the most frequent cause of the errors listed in section 10.

Step 3. Install PyTorch

This is the version used to train the published models. The download is about 2 GB.

```
python -m pip install torch==1.12.1+cu113 torchvision==0.13.1+cu113 --extra-index-url
      https://download.pytorch.org/whl/cu113
```

On a machine without an NVIDIA GPU, install the CPU build instead.

```
python -m pip install torch==1.12.1 torchvision==0.13.1
```

Step 4. Install the OpenMMLab framework

Run these three commands in order. The version numbers are the ones the models were built and tested against, and must not be changed.

```
python -m pip install -U openmim
```

```
python -m mim install mmengine==0.10.7 "mmcv>=2.0.0rc4,<2.2.0"
```

```
python -m pip install mmsegmentation==1.2.2 mmdet==3.3.0 mmpretrain==1.2.0
```

The second command is the slowest. It retrieves a large pre-built package matched to the installed PyTorch and CUDA versions, and several minutes is normal.

Step 5. Install the project

Change into the code folder. The `/d` switch is required when moving to a different drive letter.

```
cd /d D:\ghaf-project\code
```

```
python -m pip install -r requirements.txt
```

```
python -m pip install -e ".[test]"
```

> **Two messages during installation are not errors.** pip may report that `opencv-python` requires `numpy>=2`, which can be ignored, since OpenCV functions with either version. pip may also list dependency conflicts among packages this project does not use. Only a line beginning `ERROR:` that halts the installation requires action.

Paths containing spaces or an ampersand

Enclose any path containing a space in double quotes. Quotes are mandatory for a path containing an ampersand, because the Command Prompt otherwise reads `&` as the end of one command and the start of another, and reports that the path cannot be found.

```
cd /d "Z:\Survey Data\Cineraria_Data & Model\ghaf-project\code"
```

4. Verifying the installation

Three checks take five minutes between them. Run them after installation, and again whenever a result looks wrong. Together they establish that the software, the weights and the tiles are intact and agree with one another.

Check 1. The software

No GPU and no data are required. About one minute.

```
python -m pytest tests\ -q
```

Listing 5. *Expected output*

```
........................................................................
........................................................................
325 passed, 1 skipped in 74.19s
```

A final line reporting passed tests, with no `F` characters in the progress output, indicates success. The message `No module named 'mmengine'` means that a different Python is running than the one the packages were installed into. Section 10 gives the remedy.

Check 2. The models

This builds all six models from their configurations, loads the supplied weights, compares each file against its published fingerprint, and runs one prediction through each model. About 90 seconds on a GPU.

```
python tools\smoke_test.py --checkpoints D:\ghaf-project\models
```

Listing 6. *Expected output*

```
model                           digest                   weights  prediction
------------------------------------------------------------------------------
fastvit-ma36_mask2former             ok          all 1,558 matched    0.00% ghaf
poolformer-s36_fpn                   ok            all 436 matched    0.00% ghaf
dpn98_fpn                            ok            all 676 matched    0.00% ghaf
convnext-small_upernet               ok            all 430 matched    0.00% ghaf
resnet-50_mask2former                ok            all 610 matched    0.00% ghaf
efficientnet-b3_fpn                  ok            all 610 matched    0.00% ghaf

all 6 model(s) built and ran a forward pass
```

Table 6. How to read the verification table

Column	What it establishes
`digest` reads ok	The checkpoint file is identical to the one that was released. No corruption occurred during copying
`all N matched`	Every weight in the file was placed in the model built from the configuration. Nothing is missing and nothing is left over
`+0` in the delta column	The parameter count matches the published figure exactly
`0.00% ghaf`	The expected result. This check predicts on a blank synthetic tile rather than on imagery, so an absence of trees is correct. Real imagery yields a few percent

Every command that loads a model prints numerous `UserWarning` lines concerning `__floordiv__`, `torch.meshgrid`, binary segmentation and `build_loss`. These originate inside PyTorch and mmsegmentation, appear on a correct run, and require no action. The result is the table printed after them.

Check 3. The tiles

```
python tools\check_dataset.py D:\ghaf-project\data\ghaf
```

Listing 7. Expected output

```
split          images    masks   paired  checked  status
------------------------------------------------------------
training         7005     7005     7005      200  ok
validation        869      869      869      200  ok
testing           767      767      767      200  ok

8,641 paired tile(s) across 3 split(s)
dataset looks usable
```

Table 7. Options for check_dataset.py

Variant	Effect
`--full`	Opens all 8641 tiles rather than a sample of 200. Slower, and worth running once
`--sample 0`	Confirms only that every image has a matching mask, without opening any file. Completes in seconds, including over a network drive
`--sample 25`	Opens 25 tiles per split. A reasonable check after copying the data to a new location

5. Mapping one orthomosaic

This procedure takes a UAV orthomosaic and returns a canopy map. The image may be far larger than the memory of the computer. It is read in overlapping windows, and the predictions are blended where the windows meet, so the result carries no visible seams.

> **Begin with the sample clip.** `Kalba26_sample.tif` is a piece cut from the full survey. It completes in a few minutes and produces the same outputs as a full mosaic, which establishes that the installation is correct before a run of several hours is started.

5.1 The command

Change into the code folder first, which keeps the paths short.

```
cd /d D:\ghaf-project\code
```

```
python -m ghaf.inference.large_image
      ..\models\fastvit-ma36_mask2former\fastvit-ma36_mask2former.py
      ..\models\fastvit-ma36_mask2former\best_mIoU_iter_3500.pth ..\samples\Kalba26_sample.tif
      --out-mask ..\output\crowns.tif --out-prob ..\output\probability.tif --out-polygons
      ..\output\crowns.gpkg --batch-size 4
```

The arguments are, in order, the configuration, the weights, the input image, and the destination of each output. `--batch-size 4` processes four windows at a time and is faster on a GPU with memory to spare.

Listing 8. *Expected output*

```
INFO Kalba26_sample.tif: 8192 x 8192 px, 225 window(s) of 1024 px (overlap 512), batch 4, 0.6 GB
      scratch
Kalba26_sample: 100%|##############################| 57/57 [00:40<00:00,  1.42batch/s]
INFO Created 27 records
INFO wrote ..\output\crowns.gpkg (27 polygon(s))
INFO canopy: 650055 of 67108864 valid px (0.97%)
```

Between 40 seconds and two and a half minutes on one GPU, depending on how many outputs are requested and whether they are written to a local disk or a network drive.

5.2 The three outputs

Table 8. *The three outputs of an inference run*

File	Content	Typical use
`crowns.gpkg`	The crowns as polygons, with an `area_m2` column	Counting trees, measuring crowns, and joining to other GIS layers. This is the primary result
`crowns.tif`	A raster the size of the input in which each pixel is 1 for crown and 0 for background	Overlaying on the imagery, computing canopy cover, and differencing against a labelled mask
`probability.tif`	A raster of the same size holding the confidence of each pixel between 0.00 and 1.00	Applying a stricter or looser cut-off after the run, and identifying where the model was uncertain

All three carry the coordinate reference system of the input, so they align with other layers without manual positioning.

5.3 Confirming that the result is plausible

Table 9. *Values obtained on the sample clip, for comparison*

Quantity	Expected on the sample clip	If the value is far from it
Canopy percentage	About 1 per cent. Scattered Ghaf in arid terrain gives a low figure	0.00 per cent means nothing was found and a figure above 50 per cent means everything was. Both indicate a problem upstream, most often the wrong checkpoint or bands that are not red, green and blue
Polygon count	27 at the default settings	A much larger count is usually caused by single-pixel fragments. See `--min-area` below
Crown area	Between 2 and 112 square metres	Areas in the hundreds of square metres indicate that neighbouring crowns have merged, or that the imagery is not at the resolution the model expects

5.4 Options

Table 10. *Options for area-wide inference*

Option	Effect
`--threshold 0.6`	Raises the confidence required to call a pixel a crown, giving fewer crowns. A value of 0.4 is more inclusive. The default is 0.5
`--min-area 1`	Discards crown polygons below 1 square metre. This removes the single-pixel fragments that any threshold produces, which otherwise inflate the crown count. On the sample clip it removes 11 of the 27 polygons and retains the 16 genuine crowns
`--batch-size 4`	Processes four windows at a time. Reduce it to 1 if the run reports insufficient GPU memory
`--device cpu`	Runs without a GPU
`--scratch-dir E:\scratch`	Places the temporary working files on a larger or faster drive
`--bands 1 2 3`	Identifies which bands of the input are red, green and blue. Required only for imagery in an unusual band order

5.5 The full survey mosaic

`Kalba26.tif` measures 84 072 by 103 691 pixels, which is 8.7 billion pixels. At 9 bytes per pixel the run requires about 79 GB of temporary space and about 33 500 windows, giving a duration of hours rather than minutes on one GPU. Direct `--scratch-dir` at a drive with sufficient room. Free space is verified before the run begins rather than part-way through.

5.6 Cutting a clip from a larger mosaic

To test a different part of a mosaic, or to produce a quick sample from a new survey, cut a clip first. One to two minutes.

```
python tools\make_sample.py ..\samples\Kalba26.tif --output ..\samples\my_sample.tif --size 8192
      --origin 30000 40000
```

`--size` gives the clip in pixels and `--origin` its top-left corner. Omitting `--origin` takes the clip from the centre. The program also reports how much of the clip is imagery rather than the transparent border that surrounds a survey, so a badly placed window is obvious at once.

6. Mapping every image in a folder

A survey often arrives as a set of images rather than a single mosaic. This procedure maps all of them in one run. Each image is processed in windows, exactly as a full mosaic is, so the images need not share a size and none of them has to fit in memory.

```
cd /d D:\ghaf-project\code
```

```
python tools\predict_folder.py ..\models\fastvit-ma36_mask2former\fastvit-ma36_mask2former.py
      ..\models\fastvit-ma36_mask2former\best_mIoU_iter_3500.pth D:\my-images --out-dir
      ..\output\folder --batch-size 4 --min-area 1
```

Listing 9. *Expected output*

```
INFO 1 image(s) in ..\samples -> ..\output\folder
INFO [1/1] Kalba26_sample.tif
INFO Kalba26_sample.tif: 8192 x 8192 px, 225 window(s) of 1024 px (overlap 512), batch 4, 0.6 GB
      scratch
INFO dropped 11 polygon(s) under 1 m2
INFO wrote ..\output\folder\polygons\Kalba26_sample.gpkg (16 polygon(s))
INFO canopy: 650055 of 67108864 valid px (0.97%)
INFO 1 predicted, 0 skipped, 0 failed; canopy 0.97% of valid px
INFO wrote ..\output\folder\summary.json
```

6.1 The output folder

Listing 10. *Layout of the output*

```
folder\
+-- polygons\      one GeoPackage of crowns per image, with area_m2
+-- masks\         the 0/1 raster, written only with --save-mask
+-- probability\   the confidence raster, written only with --save-probability
+-- summary.json   every image, its canopy share, and any failures
```

The output of a batch run is the crowns. A GeoPackage occupies a few hundred kilobytes; the mask it was traced from occupies hundreds of megabytes. Counting, measurement and display all work from the crowns. The mask is still produced, because the polygons are traced from it, but it goes to temporary space and is deleted once that image is complete. Add `--save-mask` to keep it.

If the input folder contains subfolders, the output reproduces that structure. Two images of the same name in different subfolders cannot overwrite one another.

6.2 Options for a long run

Table 11. *Options for a folder run*

Option	Effect
`--limit 3`	Stops after three images. Run a large folder this way first, to confirm the settings before committing to hundreds of images
`--recursive`	Includes images held in subfolders
`--pattern "*_rgb.tif"`	Selects only files whose name matches. Useful where a folder mixes imagery with elevation models

Option	Effect
`--skip-existing`	Resumes an interrupted run rather than repeating images already completed
`--save-mask`	Retains the 0/1 raster for each image
`--save-probability`	Retains the confidence raster for each image
`--min-area 1`	Discards crowns below one square metre

> **A single unreadable file does not halt the run.** The image is reported, the batch continues, and the failure is recorded in `summary.json`. Read that file after any long run, since it names everything that did not succeed.

7. Reviewing the results

Every file these programs write carries the coordinate reference system of the image it came from. No manual positioning is needed.

7.1 Opening the crowns in QGIS

1. Select **Layer** > **Add Layer** > **Add Vector Layer**, browse to `crowns.gpkg`, and click **Add**.
2. Add the imagery through **Layer** > **Add Layer** > **Add Raster Layer**, then drag it below the crowns in the Layers panel.
3. Right-click the crowns layer and select **Properties** > **Symbology**. Set the fill style to *No brush* and choose a bright outline colour, so that the imagery remains visible inside each crown.
4. Right-click the layer and select **Open Attribute Table**. Each row is one crown, with its area in the `area_m2` column. The row count shown at the top of the window is the number of crowns.
5. For summary figures, select **Vector** > **Analysis Tools** > **Basic Statistics for Fields**, then choose the crowns layer and the `area_m2` field. The result gives the count, sum, mean, minimum and maximum crown area.

7.2 Opening the rasters in QGIS

Both rasters are added through **Add Raster Layer**. Two points avoid confusion.

- The mask appears black on first opening, because its values are 0 and 1 while QGIS stretches the display across 0 to 255. In **Properties** > **Symbology**, set the render type to **Paletted/Unique values** and click **Classify**, which assigns one colour to each class.
- For the probability raster, select **Singleband pseudocolor** with a colour ramp from 0 to 1. The upper end of the ramp marks the pixels the model was most confident about.

7.3 Reference figures

The values below were obtained from the delivered models and sample data. A result differing from them by an order of magnitude should be investigated before it is reported.

Table 12. *Reference figures from the delivered models and sample data*

Quantity	Value
Canopy share, sample clip	0.97 per cent of valid pixels
Canopy share, test split	3.44 per cent across the 767 test tiles
Crown count, sample clip	27 before filtering, 16 after `--min-area 1`
Crown area, sample clip	2.4 to 112 square metres
Ground sampling distance	2.68 cm per pixel
Total crown area, sample clip	469 square metres from the polygons, against 467 square metres computed from the mask pixels

8. Evaluating a model

Scoring a delivered model against the labelled test split reproduces the published figures, and is the strongest single confirmation that an installation is correct. About three minutes on a GPU.

```
cd /d D:\ghaf-project\code
```

```
python tools\test.py ..\models\fastvit-ma36_mask2former\fastvit-ma36_mask2former.py
      ..\models\fastvit-ma36_mask2former\best_mIoU_iter_3500.pth --data-root ..\data\ghaf
```

The final line reports mIoU, mDice and mFscore, above which a table gives the two classes separately. Expected values for all six models are listed in section 1.2. A departure larger than a rounding difference is caused either by a data root pointing at an unintended folder or by a checkpoint that is not the one it is taken to be, and check 2 in section 4 settles the second case.

8.1 Predictions for every tile in a split

Scoring reduces a split to a small number of figures. The maps behind those figures are sometimes wanted, for illustration or to locate the model's errors rather than measure them. Predicting the split tile by tile produces one map per tile, and takes a few minutes for the test split.

```
python tools\predict_split.py ..\models\fastvit-ma36_mask2former\fastvit-ma36_mask2former.py
      ..\models\fastvit-ma36_mask2former\best_mIoU_iter_3500.pth --data-root ..\data\ghaf
      --split testing --out-dir ..\output\predictions --save-probability
```

The result is one predicted mask per tile, encoded exactly as the ground-truth masks are, with `0` for background and `1` for ghaf, so that a prediction and its label can be subtracted directly to produce an error map. `--limit 20` performs a partial run first. The canopy fraction over the test split is 3.4 per cent.

9. Training and fine-tuning

Six trained models are supplied, so training is needed only when something is to be changed. A full run of the published configuration takes many hours on one GPU.

9.1 Training from scratch

```
python tools\train.py configs\ghaf\fastvit-ma36_mask2former.py --data-root ..\data\ghaf
```

Checkpoints and logs are written to `work_dirs\<config-name>\`. The model is scored on the validation split every 3500 iterations and the best result is retained as `best_mIoU_iter_*.pth`. An interrupted run is continued with `--resume`, which restores the optimiser state and the iteration count.

```
python tools\train.py configs\ghaf\fastvit-ma36_mask2former.py --data-root ..\data\ghaf --resume
```

> **Retain `work_dirs\` after a run completes.** It holds the loss curves, the validation history and the exact configuration used. These records cannot be reconstructed afterwards and are the material a reviewer requests.

9.2 Fine-tuning on labels from a new site

Starting from a released checkpoint costs far less than training from scratch, and usually gives a better result. Prepare the new tiles in the layout of section 1.3, as 1024 by 1024 PNG pairs whose masks contain only the values 0 and 1. Verify them before starting.

```
python tools\check_dataset.py D:\ghaf-project\data\new-site --full
```

Fine-tune with a shorter schedule and a smaller learning rate.

```
python tools\train.py configs\ghaf\fastvit-ma36_mask2former.py --data-root ..\data\new-site
      --load-from ..\models\fastvit-ma36_mask2former\best_mIoU_iter_3500.pth --cfg-options
      train_cfg.max_iters=4000 optim_wrapper.optimizer.lr=1e-5
```

Table 13. *Arguments used when fine-tuning*

Argument	Effect
`--load-from`	Takes the weights and begins a new schedule, which is what fine-tuning requires
`--resume`	Continues an interrupted run of your own, retaining the optimiser state and iteration count. Distinct from `--load-from`
`--cfg-options train_cfg.max_iters=4000 optim_wrapper.optimizer.lr=1e-5`	Shortens the schedule to 4000 iterations and lowers the learning rate, so that the model adapts to the new site without discarding what it has already learned

Score the result as described in section 8, with `--data-root` directed at the new site and the checkpoint at the new `work_dirs\...\best_mIoU_iter_*.pth`.

10. Diagnosing errors

No command in this handbook deletes data, and an interrupted run can be started again. Nearly every failure announces itself in the last few lines of output. This section explains how to read them.

10.1 Procedure

1. **Read the last three lines.** Python prints a record of what it was doing and then, on the final line, what went wrong. That final line is the error; everything above it is context.
2. **Identify the error type.** The final line takes the form `SomeError: message`. The word before the colon states the category, for example `FileNotFoundError`, `RuntimeError` or `ModuleNotFoundError`.
3. **Consult the table in section 10.3**, which covers the failures that occur in practice.
4. **Answer the three questions in section 10.2**, which resolve a large proportion of problems without further investigation.
5. **Search**, following section 10.4.

> **Copy the error text before running anything else.** Select it in the terminal window, press Enter to copy, and paste it into a text file. Closing the window or running another command usually loses it, and an error that cannot be quoted cannot be diagnosed by anyone else.

10.2 Three questions to answer first

Table 14. *Checks to make before investigating further*

Question	How to check	Why it matters
Is the `ghaf` environment active?	The prompt begins with `(ghaf)`. If it does not, run `conda activate ghaf`	This is the most frequent cause of a missing module. Every new terminal window opens outside the environment
Is the correct Python running?	Run `python -c "import sys; print(sys.executable)"`. The path printed must contain `envs\ghaf`	A base Anaconda installation earlier in the PATH answers instead, and holds none of this project's packages
Is the working folder correct?	Run `cd` alone to print the current folder, and `dir` to list its contents	The commands in this handbook assume the `code` folder. A relative path entered elsewhere will not resolve

10.3 Reported errors and their remedies

Table 15. *Reported errors and their remedies*

Message	Cause	Remedy
`No module named mmengine`, `pytest` or `ghaf`	A different Python is running than the one the packages were installed into	Run `conda activate ghaf` and confirm with `python -c "import sys; print(sys.executable)"`. Begin commands with `python -m`

Message	Cause	Remedy
`mmseg ... is not installed in conda environment "X"`	The wrong environment is active, or the framework was never installed on this machine	The message names the interpreter that was queried. Run `conda activate ghaf` and repeat the command
`No module named ftfy`	mmsegmentation imports a tokenizer requiring this package, although its own metadata does not declare it	Run `python -m pip install ftfy regex`
`RuntimeError: Numpy is not available`	NumPy 2 is installed alongside a PyTorch built against NumPy 1	Run `python -m pip install "numpy<2"`
`opencv-python ... requires numpy>=2` during installation	A note from the pip resolver rather than an error	No action. OpenCV functions with either version
`CUDA out of memory`	The GPU has insufficient free memory, commonly because the batch size is too high or another program is using the card	Reduce `--batch-size` to 2 or 1, close other GPU programs, or add `--device cpu`
`not enough scratch space`	The drive holding temporary files is too small for this image	Add `--scratch-dir` directed at a drive with room. An image requires 9 bytes per pixel
`FileNotFoundError` naming a path	A file or folder in the command does not exist as entered	Check for a typographical error, a missing `..\`, or a path with spaces requiring double quotes
`The system cannot find the path specified`, printed twice	The path contains an ampersand, which the Command Prompt read as two separate commands	Enclose the whole path in double quotes
`The process cannot access the file because it is being used by another process`	QGIS, ArcGIS or another program holds the file open	Close the program concerned. When deleting a folder, first change out of it, since a terminal within a folder holds it open
`NO TILES` from `check_dataset.py`	The folders exist but contain no PNG or TIF tiles	The row names the file types found instead. The tiles are usually one level deeper, or in a different format
`SHA-256 mismatch` from `smoke_test.py`	A checkpoint does not match the file that was released	That copy is damaged. Copy it again from the original
`0.00% ghaf` from `smoke_test.py`	None. That check predicts on a blank tile	No action. See section 4
0.00 per cent canopy on real imagery	The run found nothing	Usually the wrong checkpoint, or bands that are not red, green and blue. Verify the configuration and checkpoint paths, then try `--bands`
Numerous `UserWarning` lines about `__floordiv__`, `meshgrid` or `build_loss`	Deprecation notices from within PyTorch and mmsegmentation	No action. They appear on a correct run

Message	Cause	Remedy
`KeyboardInterrupt`	Ctrl+C was pressed, or the terminal was closed	Nothing is damaged. Run the command again. Long runs resume with `--skip-existing` or `--resume`
Unexpected behaviour after pasting several lines	The terminal joined them into a single line	Enter one command at a time

10.4 Searching for a solution

An error absent from the table above has been met by someone else already. The procedure below finds their answer.

Preparing the query

Take the final line of the error. Strip out everything specific to your computer, meaning the user name, drive letters, file names and long numbers. None of it appears in anyone else's error. Add the name of the library that raised the failure.

Listing 11. *The error as printed*

```
  File "D:\ghaf-project\code\ghaf\inference\large_image.py", line 322, in predict_large_image
    with rasterio.open(src_path) as src:
rasterio.errors.RasterioIOError: D:\surveys\flight_07\north_block.tif: No such file or directory
```

Listing 12. *The text to search for*

```
rasterio RasterioIOError No such file or directory
```

Where to search, in order

Table 16. *Where to search, in order of preference*

Source	Method	Best suited to
A search engine	Enter the prepared query, with the name of the library that raised the error, such as `mmsegmentation`, `rasterio` or `pytorch`	Most problems. Start here
The library issue tracker	Open the library page on GitHub, select **Issues**, and search there. Include closed issues, since a closed issue commonly contains the resolution	Errors raised inside mmsegmentation, mmcv or mmdet, at github.com/open-mmlab/mmsegmentation
Stack Overflow	Search the error text and read the accepted answer together with the comments beneath it	General Python, conda and installation problems
The project documentation	The `docs\` folder beside the code, which holds GETTING_STARTED, AREA_WIDE_INFERENCE, MODEL_ZOO and RELEASE_BUNDLE	Questions about this project rather than the libraries it uses
The program itself	Append `--help` to any command, for example `python tools\predict_folder.py --help`	The name and effect of an option

Assessing what you find

- **Check the versions.** An answer written for mmsegmentation 0.x does not apply to version 1.2.2, and one written for PyTorch 2.x may not apply to 1.12.1. Treat a page that does not state its versions with caution.
- **Prefer answers that explain the cause.** An answer offering a command without stating what it corrects is as likely to cause damage as to help.
- **Do not upgrade packages on the strength of general advice.** Instructions to upgrade mmcv, PyTorch or NumPy will break this installation. The versions given in section 3 were chosen to work together and to match the trained weights. If a version must be changed, record the existing one first.
- **Change one thing at a time**, and repeat the checks in section 4 after each change. Two simultaneous changes leave the cause of any improvement unknown.

> **Reinstalling.** If an installation reaches a state that cannot be explained, delete the environment and rebuild it from section 3. This takes about twenty minutes and affects no data. Run `conda deactivate`, then `conda env remove -n ghaf`, then begin at step 2.

10.5 Reporting a problem to a colleague

Provide all six items below. A complete report is normally resolved in one exchange rather than several.

1. The exact command, copied rather than retyped.
2. The complete output, from the command to the final line, as text rather than as an image.
3. The result that was expected instead.
4. The output of `python -c "import sys; print(sys.executable)"`.
5. The output of `python -m pip list`, or at minimum the entries for `torch`, `mmcv`, `mmsegmentation`, `mmdet` and `numpy`.
6. Whether the checks in section 4 pass at present, and whether they have ever passed on this machine.

11. Command reference

Every routine command, collected in one place. Each assumes that `conda activate ghaf` and `cd /d D:\ghaf-project\code` have been run in the same window, that `MODEL` stands for `..\models\fastvit-ma36_mask2former\fastvit-ma36_mask2former.py`, and that `WEIGHTS` stands for `..\models\fastvit-ma36_mask2former\best_mIoU_iter_3500.pth`.

Verification

```
python -m pytest tests\ -q
python tools\smoke_test.py --checkpoints ..\models
python tools\check_dataset.py ..\data\ghaf
```

Inference

```
python -m ghaf.inference.large_image MODEL WEIGHTS image.tif --out-polygons crowns.gpkg
      --out-mask crowns.tif --batch-size 4 --min-area 1

python tools\predict_folder.py MODEL WEIGHTS D:\my-images --out-dir ..\output\folder
      --batch-size 4 --min-area 1

python tools\make_sample.py big-mosaic.tif --output sample.tif --size 8192
```

Evaluation and training

```
python tools\test.py MODEL WEIGHTS --data-root ..\data\ghaf

python tools\predict_split.py MODEL WEIGHTS --data-root ..\data\ghaf --split testing --out-dir
      ..\output\predictions

python tools\train.py configs\ghaf\fastvit-ma36_mask2former.py --data-root ..\data\ghaf

python tools\train.py configs\ghaf\fastvit-ma36_mask2former.py --data-root ..\data\new-site
      --load-from WEIGHTS --cfg-options train_cfg.max_iters=4000 optim_wrapper.optimizer.lr=1e-5
```

Diagnosis

```
conda activate ghaf
python -c "import sys; print(sys.executable)"
python tools\predict_folder.py --help
conda env remove -n ghaf
```

Further documentation

Table 17. *Further documentation*

Document	Subject
`code\README.md`	The project, its results, and the repository layout
`code\docs\GETTING_STARTED.md`	The same procedures as this handbook, in the code folder
`code\docs\MODEL_ZOO.md`	All six models, per-class scores and training settings
`code\docs\AREA_WIDE_INFERENCE.md`	How a mosaic is divided into windows and blended, and how to adjust it
`code\docs\RELEASE_BUNDLE.md`	How the models folder is assembled and verified
`MANIFEST.json`	The contents of the bundle, its sizes, and the version of the code that produced it

This handbook describes the repository at github.com/brakuta/Ghaf-Tree-Crown-Mapping-from-UAV-Data. The trained weights, the UAV imagery and the labelled tiles are not distributed with the code and are available from the corresponding author on reasonable request.